

Integration der Trajektorien-Ermittlung in das Livesystem

Dieses Kapitel beschreibt, wie das Modul `trajectory.py` in ein Livesystem integriert werden kann. Ziel ist es, eingehende Positionsdaten zu sammeln, daraus eine gültige JSON-Datei zu erzeugen und anschließend die Trajektorien-Ermittlung auszuführen.

Die Entscheidung, **wo** die Berechnung ausgeführt wird, muss von der nachfolgenden Gruppe getroffen werden. Dieses Kapitel beschreibt mögliche Integrationsvarianten und die jeweiligen Anforderungen.

Voraussetzungen

Die Funktion `determine_tracks(records)` erwartet eine Liste von Beobachtungen. Jede Beobachtung muss mindestens folgende Felder enthalten:

Feld	Datentyp	Beschreibung
<code>timestamp</code>	String	Zeitpunkt der Beobachtung im ISO-8601-Format
<code>enu_e</code>	Zahl	Ost-Koordinate im ENU-Koordinatensystem
<code>enu_n</code>	Zahl	Nord-Koordinate im ENU-Koordinatensystem
<code>enu_u</code>	Zahl	Höhen-Koordinate im ENU-Koordinatensystem

Die verwendeten Koordinaten müssen innerhalb eines Datensatzes dieselbe Einheit verwenden. Die Parameter `MAX_SPEED_UNITS_PER_SEC` und `DISTANCE_MARGIN` müssen zu dieser Einheit passen.

Beispiel: Werden ENU-Koordinaten in Metern gespeichert, entspricht `MAX_SPEED_UNITS_PER_SEC = 25` einer maximalen Geschwindigkeit von 25 m/s.

Erwartetes JSON-Eingabeformat

Die aktuelle Implementierung liest die Beobachtungen aus dem Feld `simulated_birds`. Die JSON-Datei muss daher folgende Struktur haben:

```

{
  "simulated_birds": [
    {
      "timestamp": "2026-09-09T10:15:00.000Z",
      "enu_e": 125.4,
      "enu_n": 87.2,
      "enu_u": 14.8
    },
    {
      "timestamp": "2026-09-09T10:15:01.000Z",
      "enu_e": 130.1,
      "enu_n": 88.6,
      "enu_u": 15.2
    },
    {
      "timestamp": "2026-09-09T10:15:01.000Z",
      "enu_e": 420.7,
      "enu_n": 302.4,
      "enu_u": 21.0
    }
  ]
}

```

Mehrere Beobachtungen dürfen denselben Zeitstempel besitzen. Das Modul verarbeitet diese gemeinsam und versucht, sie bestehenden Trajektorien zuzuordnen.

Zusätzliche Felder sind erlaubt. Sie werden von der Berechnung nicht benötigt, bleiben jedoch im Ergebnis erhalten. Beispielsweise können Informationen über Sensor, Kamera oder Raw-Daten mitgespeichert werden. Dies würde aber die Übersichtlichkeit der JSON beeinträchtigen.

Datensammlung im Livesystem

Damit die Funktion verwendet werden kann, muss die Pipeline des Livesystems eingehende Events in einer JSON-Struktur sammeln.

Bei jedem relevanten Event sollte eine Beobachtung erzeugt und an die Sammlung angehängt werden.

Der grundsätzliche Ablauf lautet:

1. Ein Sensor- oder Detektionseignis trifft im Livesystem ein.
2. Die für die Trajektorien-Ermittlung erforderlichen Daten werden aus dem Event extrahiert.

3. Geografische Positionsdaten werden in ENU-Koordinaten umgerechnet.
4. Es wird ein Beobachtungsobjekt mit `timestamp`, `enu_e`, `enu_n` und `enu_u` erzeugt.
5. Dieses Objekt wird an `simulated_birds` in der Sammel-JSON angehängt.
6. Nach Abschluss der Datensammlung wird die JSON-Datei an `trajectory.py` übergeben.

Beispielhaft kann eine fortlaufend aufgebaute JSON-Struktur wie folgt aussehen:

```
{
  "simulated_birds": [
    {
      "timestamp": "2026-09-09T10:15:00.000Z",
      "enu_e": 125.4,
      "enu_n": 87.2,
      "enu_u": 14.8
    },
    {
      "timestamp": "2026-09-09T10:15:01.000Z",
      "enu_e": 130.1,
      "enu_n": 88.6,
      "enu_u": 15.2
    }
  ]
}
```

Wichtig ist, dass die JSON-Datei während der Sammlung syntaktisch gültig gespeichert wird. Bei vielen Events oder längeren Laufzeiten kann alternativ eine Datenbank oder ein Event-Log verwendet werden. Vor der Berechnung werden die gesammelten Datensätze dann in das erwartete JSON-Format exportiert.

Zeitpunkt der Ausführung

Die vorliegende Implementierung verarbeitet die Beobachtungen als vollständige Datenmenge. Sie sortiert alle Beobachtungen zeitlich und bewertet sie anschließend schrittweise.

Daher sollte die Funktion standardmäßig **erst nach dem Ende der Datensammlung** ausgeführt werden.

Mögliche Auslöser für die Verarbeitung sind:

- Ende einer Simulation,
- Ende eines definierten Zeitfensters,
- manueller Start durch einen Benutzer,

- Abschluss einer Messung,
- geplanter Hintergrundjob, beispielsweise alle fünf Minuten.

Eine direkte Verarbeitung bei jedem einzelnen Event ist mit der aktuellen Umsetzung nicht vorgesehen. Dafür müsste die Funktion erweitert werden, damit aktive Tracks, letzte Positionen und Geschwindigkeiten dauerhaft gespeichert und bei neuen Events wieder geladen werden können.

Betriebsart	Beschreibung	Anpassung erforderlich
Nachgelagerte Batch-Verarbeitung	Alle Events werden gesammelt, anschließend wird die Trajektorie berechnet.	Nein
Zeitfenster-Verarbeitung	Daten werden beispielsweise alle fünf Minuten gesammelt und gemeinsam verarbeitet.	Geringfügig
Live-Verarbeitung	Neue Events aktualisieren bestehende Tracks unmittelbar.	Ja, zustandsbehaftete Erweiterung erforderlich

Variante 1: JSON-Download und lokale Ausführung

Bei dieser Variante sammelt das Livesystem die Beobachtungen und stellt die daraus erzeugte JSON-Datei im Webinterface zum Download bereit.

Die Benutzer können die Datei anschließend auf einem lokalen Rechner mit ausreichender Rechenleistung verarbeiten.

Ablauf:

1. Das Livesystem sammelt Positionsdaten.
2. Im Webinterface wird ein Download der JSON-Datei angeboten.
3. Die Datei wird lokal gespeichert.
4. Das Skript `trajectory.py` wird lokal ausgeführt.
5. Die Ergebnisdatei mit den ermittelten Track-IDs wird lokal und kann verarbeitet werden.

Vorteile:

- Keine zusätzliche Rechenlast auf dem Livesystem.
- Berechnung kann auf leistungsfähiger Hardware stattfinden.
- Geringerer Integrationsaufwand im Livesystem.

- Gut geeignet für große Datenmengen oder Entwicklungs- und Analysezwecke.

Nachteile:

- Manueller Zwischenschritt notwendig.
- Ergebnisse sind nicht automatisch im Livesystem sichtbar.
- Ein lokaler Python-Client oder eine lokale Python-Umgebung wird benötigt.

Die Dateinamen `INPUT_FILE` und `OUTPUT_FILE` müssen dabei entweder im Skript angepasst oder zukünftig über Programmargumente konfigurierbar gemacht werden.

Variante 2: Ausführung direkt im Livesystem

Bei dieser Variante wird eine zusätzliche Seite oder Funktion im Webinterface des Livesystems bereitgestellt. Nach Abschluss der Datensammlung kann ein Benutzer die Trajektorien-Ermittlung direkt im System starten.

Ablauf:

1. Das Livesystem sammelt die Beobachtungen.
2. Ein Benutzer öffnet eine Seite zur Trajektorien-Ermittlung.
3. Der Benutzer startet die Berechnung.
4. Das Livesystem übergibt die gesammelten Daten an `determine_tracks`.
5. Die Ergebnisse werden gespeichert.
6. Die Trajektorien oder Track-IDs werden im Webinterface angezeigt.

Mögliche Anzeigeformen:

- Tabelle mit Beobachtungen und zugehöriger `determined_track_id`,
- Darstellung der Flugbahnen auf einer Karte oder in einer 3D-Ansicht,
- Download der Ergebnis-JSON-Datei,
- Filterung nach Track-ID und Zeitbereich,
- Anzeige von Anzahl, Dauer und räumlichem Verlauf der Tracks.

Vorteile:

- Keine lokale Installation notwendig.
- Ergebnisse stehen unmittelbar im Livesystem zur Verfügung.
- Einheitlicher Arbeitsablauf für Benutzer.
- Die Ergebnisse können direkt mit weiteren Live-System-Daten kombiniert werden.

Nachteile:

- Das Livesystem benötigt ausreichende Rechenkapazität.
- Die Ausführung darf das laufende System nicht blockieren.
- Für große Datenmengen sollten Hintergrundjobs, Warteschlangen oder Begrenzungen vorgesehen werden.

Für diese Variante sollte die Berechnung nicht direkt im Request des Webinterfaces stattfinden. Stattdessen empfiehlt sich ein Hintergrundprozess. Das Webinterface startet dabei einen Auftrag und zeigt anschließend dessen Status an.

Variante 3: Separater Client für die Trajektorien-Ermittlung

Eine weitere Möglichkeit ist ein separater Client. Dieser kann die Funktion `determine_tracks` enthalten und Daten über eine Schnittstelle aus dem Livesystem abrufen.

Ablauf:

1. Das Livesystem stellt Beobachtungen über Download oder API bereit.
2. Ein externer Client lädt die Daten herunter.
3. Der Client führt die Trajektorien-Ermittlung lokal oder auf einem separaten Server aus.
4. Der Client speichert oder visualisiert die Ergebnisse.
5. Optional werden die Ergebnisse über eine API wieder an das Livesystem übertragen.

Diese Variante verbindet die Vorteile einer leistungsfähigen externen Verarbeitung mit einer möglichen späteren Anzeige im Livesystem.

Entscheidungspunkte für die nächste Gruppe

Vor der Implementierung sollte die nächste Gruppe insbesondere folgende Punkte entscheiden:

1. Soll die Berechnung als Batch-Verarbeitung, in Zeitfenstern oder vollständig live erfolgen?
2. Reicht die Rechenleistung des Livesystems für die erwartete Anzahl an Beobachtungen aus?
3. Sollen Nutzer JSON-Dateien herunterladen oder Ergebnisse direkt im Webinterface berechnen und anzeigen lassen?
4. Wie lange werden Rohdaten gesammelt und gespeichert?
5. Soll ein separater Client oder externer Rechenserver eingesetzt werden?

6. Sollen berechnete `determined_track_id` -Werte wieder im Livesystem gespeichert werden?

Die aktuell vorhandene Implementierung eignet sich unmittelbar für eine Verarbeitung nach abgeschlossener Datensammlung. Eine echte Live-Verarbeitung ist möglich, erfordert jedoch eine Erweiterung um dauerhaft gespeicherte Track-Zustände.